

LLM 버그바운티 자동화 파이프라인

v5.0 최종 기능 버전 개발 계획서

현재 기준: v4.4.0 | 최종 기능 동결: v5.0.0

2026-09-24

목표

기능을 무한히 확장하지 않고, 정해진 범위의 취약점을
탐지 - 안전 검증 - 재현 - 증거화 - 리포팅까지 닫는 CLI 기반 도구를 완성한다.

최종 방향

v5.0.0 이후에는 새 기능 확장보다 실전 테스트에서 확인된 오류·오탐·미탐을 수정한다.

1. 계획 개요

이 문서는 v4.4.0 까지 구현된 LLM 버그바운티 자동화 파이프라인을 v5.0.0 에서 기능 동결하기 위한 구체적인 개발 계획을 정의한다. 핵심 원칙은 기능 수를 늘리는 것이 아니라, 이미 선택한 범위의 탐지와 검증 흐름을 실제 사용 가능한 수준으로 닫는 것이다.

핵심 원칙 “더 많이 지원”이 아니라 “지원한다고 선언한 범위를 끝까지 처리”한다. v5.0.0 이후에는 실전 테스트 결과를 근거로 한 수정만 우선한다.

1.1 v5.0 에서 완성해야 할 것

- LLM / RAG / Agent 프롬프트 인젝션 계열의 탐지 - 판정 - 재현 - 증거 - 리포트 흐름 유지 및 안정화
- Web/API 동적 검증을 GET 계열 중심에서 POST / PUT / PATCH / DELETE 까지 확장하되, 상태 변경 위험을 정책으로 제어
- Auth / IDOR / BOLA 후보를 실제 요청 컨텍스트와 연결하고, 안전한 비교 검증까지 수행
- Python 과 JS/TS 에서 route - auth - dataflow - sink 흐름을 최소 실용 수준으로 연결
- 존재하지만 직접 호출해야 하는 validator 를 CLI 워크플로우에 연결
- 자체 fixture 가 아닌 실제 취약 애플리케이션으로 benchmark 와 regression 기준을 확립
- Evidence / reproduce / report 산출물을 실전 버그바운티 제출 흐름에 맞게 안정화

1.2 명시적 비목표

- Java / Spring, PHP, Go, .NET, Rails 등 모든 언어와 프레임워크를 v5.0 이전에 지원하지 않는다.
- Burp Suite, CodeQL, Semgrep, Nuclei 를 완전히 대체하는 범용 보안 플랫폼을 만들지 않는다.
- 대시보드, SaaS, 팀 계정, 멀티테넌시, 중앙 서버 같은 플랫폼 기능은 구현 범위에서 제외한다.
- 파괴적 exploit 을 자동 실행해 “모든 취약점을 확정”하는 구조를 목표로 하지 않는다.
- 새로운 취약점 클래스가 생각날 때마다 자동으로 범위를 확장하지 않는다.

2. v4.4.0 현재 상태

영역	현재 수준	판단
LLM/RAG/Agent	Canary, negative control, 반복 시도/통계, Judge ensemble, reproduce	가장 실전적
Evidence/Report	raw/sanitized 분리, SHA-256 manifest, redaction	강점
Scope/Policy	실행 전 scope 검증 및 위험 동작 gating	강점
Python 정적 분석	Flask/FastAPI route/auth/dataflow 일부 AST 분석	사용 가능
JS/TS 정적 분석	Express/Next/Nest 일부, 함수 간 전파 약함	보완 필요
Web 동적 검증	GET/HEAD/OPTIONS 중심	구조적 한계
Auth/Dataflow validator	기능 존재하나 CLI 연결 미완	연결 필요
Judge	regex/canary/refusal 중심	semantic 판정 부족
GraphQL	분류 중심, 전용 로직 약함	v5.0 핵심 범위 아님
Benchmark	fixture 2 개 / finding 5 개 수준	실전 검증 부족

현재 프로젝트의 강점 LLM/RAG/Agent 취약점은 “후보 탐지”에서 끝나지 않고 재현 가능한 finding 과 evidence, report 까지 자동으로 연결할 수 있다는 점이다.

3. v5.0 최종 구조

v5.0 은 아래 파이프라인을 최종 기능 구조로 삼는다. 이후 기능 확장은 이 흐름의 품질을 실제로 개선하는 경우에 만 허용한다.

단계	역할	v5.0 완료 조건
1. Scope Check	허가 범위, host/path/method/policy 검증	모든 실행 경로가 공통 gate 를 거침
2. Static Analysis	route/auth/source/sink 후보 수집	Python + JS/TS 핵심 패턴 연결
3. Candidate Generation	테스트 가능한 finding 후보 생성	중복 제거 및 근거 포함
4. Safe Validation	실제 요청 기반 안전 검증	read-only / state-changing 정책 분리
5. Judge	규칙 + semantic 보조 판정	불확실하면 needs_review
6. Reproduction	동일 조건 재실행	재현 성공률/시도 기록
7. Evidence	원본/정제/무결성 보존	manifest 및 redaction 일관
8. Report	제출 가능한 요약/근거/재현 절차	finding 단위 산출물 완성

4. 버전 로드맵

버전	핵심 목적	완료 선언 조건
4.5	실행 경로와 API/Auth 검증 연결	상태 변경 method + Auth/BOLA + validator CLI 연결
4.6	분석 정확도와 Judge 보강	Python/JS-TS dataflow 개선 + semantic judge 보조
4.7	실전 benchmark 와 안정화	실제 취약 앱 평가 + regression + crash/timeout 정리
5.0	기능 동결 및 release hardening	문서/CLI/산출물/회귀 테스트 통과 후 freeze

5. v4.5 - API/Auth 검증 완성

v4.5의 목적은 “후보는 찾지만 실제 검증 경로가 끊기는 문제”를 줄이는 것이다. 새로운 취약점 클래스를 많이 추가하기보다, 현재 존재하는 validator와 실제 HTTP 요청을 하나의 CLI 흐름으로 연결한다.

5.1 상태 변경 HTTP Method 지원

- POST / PUT / PATCH / DELETE endpoint를 정적 분석과 endpoint inventory에 포함한다.
- JSON, form-urlencoded, multipart 등 body schema를 route/schema 정보에서 가능한 범위만 추론한다.
- 기존 캡처 요청이나 명시적 사용자 입력이 있으면 추론보다 실제 요청 구조를 우선한다.
- method 별 위험 등급을 부여하고, state-changing 요청은 dry-run / review-required / allowed 상태를 명확히 구분한다.
- 자동 검증은 기본적으로 무해한 파라미터 변경과 비교 요청에 제한한다.

5.2 Auth / IDOR / BOLA 기본 검증

- 인증 유무, 사용자 A/B 세션, role 차이를 비교할 수 있는 auth context 모델을 만든다.
- path/query/body에서 object identifier 후보를 추출한다.
- 동일 endpoint에서 “내 객체 / 다른 테스트 객체”의 응답 상태, 길이, 구조, 핵심 필드 차이를 비교한다.
- 단순 200/403 비교가 아니라 response fingerprint와 권한 관련 근거를 evidence에 저장한다.
- 권한이 다른 두 테스트 계정이 없으면 confirmed가 아니라 candidate 또는 needs_review로 남긴다.

5.3 Validator CLI 연결

CLI 흐름	예시 역할	완료 기준
scan	정적 분석 + endpoint inventory + candidate 생성	validator가 사용할 공통 artifact 생성
validate	candidate 별 safe validator 실행	auth/dataflow/endpoint validator 접근 가능
reproduce <id>	선택 finding 재실행	동일 조건/횟수/결과 저장
report	evidence 기반 보고서 생성	confirmed/needs_review 상태 반영

5.4 v4.5 완료 기준

- CLI만으로 scan -> validate -> reproduce -> report 흐름이 끊기지 않는다.
- GET 외의 method가 endpoint inventory와 validator에 실제로 전달된다.
- Auth/BOLA 테스트가 세션 컨텍스트 부족 시 안전하게 중단되고 이유를 기록한다.
- state-changing 요청이 정책 gate를 우회하지 못한다.
- 기존 LLM 취약점 테스트가 regression 없이 동작한다.

6. v4.6 - 분석 정확도와 Judge 보강

6.1 Python interprocedural dataflow

- route -> helper/service -> sink의 최소 1~N hop 호출 그래프를 구성한다.
- Flask/FastAPI decorator 기반 auth 요구사항과 dependency를 route metadata에 연결한다.
- ORM/query/raw execution, subprocess, template/render, URL fetch 등 핵심 sink family를 표준화한다.
- 분석 깊이/노드 수 cap을 뒤 대형 코드베이스에서 무한 확장을 방지한다.

6.2 JS/TS dataflow 실용화

- Express middleware -> handler -> service/wrapper 흐름을 추적한다.
- Next.js API route/server action과 NestJS controller -> service 경로를 최소 지원한다.
- 함수 간 직접 호출 및 단순 wrapper 전파를 지원하되, 복잡한 동적 dispatch는 명시적으로 미지원 처리한다.
- sink 목록은 개수를 무한히 늘리기보다 실제 benchmark에서 반복 등장하는 계열만 유지한다.

6.3 Semantic Judge 보조

- 기존 canary/regex/refusal 기반 deterministic judge 를 1 차 판정으로 유지한다.
- 정규식만으로 판단하기 어려운 정책 위반/데이터 누출에 semantic judge 를 2 차로 사용한다.
- 두 판정이 충돌하거나 confidence 가 낮으면 confirmed 로 올리지 않고 needs_review 로 보낸다.
- judge prompt/version/model 정보를 evidence metadata 에 기록해 재현성을 확보한다.

6.4 v4.6 완료 기준

- 대표 Python fixture 에서 route -> service -> sink 전파가 회귀 테스트로 고정된다.
- 대표 Express/Next/Nest fixture 에서 최소 1-hop 이상 wrapper 추적이 검증된다.
- Judge 결과가 deterministic / semantic / final verdict 세 층으로 남는다.
- 분석 시간 상한과 graph cap 이 설정되어 비정상적으로 확장되지 않는다.

7. v4.7 - 실전 Benchmark 와 안정화

v4.7 에서는 기능 추가를 사실상 중단하고, 현재 기능이 실제 취약 애플리케이션에서 얼마나 유효한지 측정한다. 이 단계부터 “아이디어”보다 “측정 결과”가 우선한다.

7.1 Benchmark 구성 원칙

대상군	목적	측정 항목
LLM 취약 앱	Prompt/RAG/Agent 측의 실전 재현성	confirmed rate, reproducibility, FP
Flask/FastAPI 취약 앱	Python route/auth/dataflow	TP/FP/FN, sink/path coverage
Express/Next/Nest 취약 앱	JS/TS 분석 실전성	wrapper miss, auth miss, endpoint coverage
Web/API 취약 앱	Auth/IDOR/BOLA 안전 검증	candidate -> validated conversion, false alarms

7.2 반드시 기록할 지표

- Finding 수가 아니라 TP / FP / FN 을 분리한다.
- 미탐 원인을 unsupported framework, dataflow cut, auth context missing, validator limitation, judge false negative, policy blocked 등으로 분류한다.
- confirmed finding 은 재현 성공 횟수와 실패 횟수를 같이 저장한다.
- 실패/timeout/crash 는 finding 품질과 별도로 안정성 지표로 기록한다.
- benchmark 는 버전별 동일 데이터셋으로 반복해 회귀를 확인한다.

7.3 수정 우선순위 규칙

수정 허용 기준 같은 미탐/오탐 원인이 여러 실전 케이스에서 반복되고, 수정이 프로젝트 핵심 범위의 품질을 명확히 높일 때만 탐지 로직을 확장한다.

- P0: crash, scope bypass, evidence 손상, 위험 요청 오분류
- P1: 반복되는 핵심 취약점 미탐, 재현 실패, CLI 흐름 단절
- P2: 높은 빈도의 오탐, 분석 성능 병목, report 품질 문제
- P3: 단일 framework/회귀 패턴/편의 기능 - v5.0 이전에는 원칙적으로 보류

7.4 v4.7 완료 기준

- 자체 fixture 외 실제 취약 앱 benchmark 결과가 저장된다.
- 주요 regression test 가 CI에서 반복 가능하다.
- 반복되는 P0/P1 문제가 모두 수정되거나 known limitation 으로 문서화된다.
- precision/recall 수치는 데이터셋 크기와 대상군을 함께 명시한다.
- 기능을 더 넣지 않아도 v5.0 으로 동결 가능한 상태가 된다.

8. v5.0 - 기능 동결과 Release Hardening

v5.0.0은 “더 이상 기능을 추가하지 않아도 된다”는 기준선이다. 이 버전에서 중요한 것은 기능 수가 아니라 일관성, 재현성, 안전성, 문서화다.

8.1 Release Hardening 항목

- CLI 명령/옵션/exit code 정리 및 help 문서 일치
- config / scope / policy schema 버전 고정
- finding 상태(candidate / needs_review / confirmed / rejected) 정의 고정
- evidence 디렉터리 구조와 manifest schema 고정
- reproduce 에서 실행 환경, adapter, judge, 횡수, 결과를 모두 남김
- report template 의 최소 필수 필드 고정
- known limitations 와 지원 matrix 공개
- P0/P1 regression suite 전체 통과

8.2 최종 지원 범위

구분	v5.0 지원	v5.0 에서 하지 않음
LLM/RAG/Agent	Prompt injection, indirect injection, leakage, tool/agent misuse 계열의 안전 검증	모든 모델/모든 정책 우회 보장
Web/API	GET + 상태 변경 method inventory/안전 검증, Auth/IDOR/BOLA 기본 비교	파괴적 exploit 자동화
Static	Python + JS/TS 핵심 framework 흐름	전 언어/전 framework AST/dataflow
Judge	규칙 기반 + semantic 보조 + needs_review	LLM 판단만으로 확정
Evidence	raw/sanitized, hash manifest, redaction	법적/조직별 제출 형식 전체 자동 대응
Interface	로컬/CLI 파이프라인	SaaS/대시보드/멀티유저 플랫폼

8.3 v5.0.0 최종 Release Gate

- ☐ 기본 설치 후 샘플 프로젝트에서 end-to-end 실행 가능
- ☐ scope 밖 대상은 실행 전에 차단
- ☐ 상태 변경 요청은 정책 없이 자동 실행되지 않음
- ☐ LLM finding 최소 대표 세트가 재현 가능
- ☐ Auth/BOLA 비교 validation 이 테스트 계정 환경에서 동작
- ☐ Python/JS-TS 대표 dataflow regression 통과
- ☐ evidence hash 검증 및 redaction 테스트 통과
- ☐ report 생성 후 finding 근거와 재현 절차가 누락되지 않음
- ☐ known limitation 문서 작성 완료
- ☐ benchmark 결과와 데이터셋 범위 명시 완료

9. 실전 테스트 운영 계획

v5.0.0 이후에는 새 기능 로드맵 대신 실제 허가된 대상에서의 실패 사례를 수집하고 수정한다. 버전 번호는 5.0.x 패치 계열을 기본으로 사용한다.

상황	처리 원칙	예시 버전
Crash / scope / integrity 버그	즉시 수정	5.0.1

상황	처리 원칙	예시 버전
반복 미탐/오탐	재현 케이스 추가 후 탐지 로직 수정	5.0.2
Report/evidence 품질 문제	schema 호환 유지하며 보완	5.0.3
새 framework 지원 요청	실전 반복 필요성이 없으면 보류	미정
대규모 구조 변경 필요	5.x에서 억지로 넣지 않고 별도 판단	6.0 검토 가능

9.1 실전 테스트 기록 템플릿

필드	기록 내용
Target	허가된 대상/로컬 취약 앱 식별자
Expected	알려진 취약점 또는 기대 탐지 결과
Observed	도구가 실제 생성한 candidate/verdict
Outcome	TP / FP / FN / Blocked / Error
Root Cause	dataflow cut, auth context, policy, judge 등
Fix Decision	fix now / document limitation / reject
Regression	수정 후 자동 테스트 케이스 추가 여부

10. 주요 리스크와 통제

리스크	영향	통제 방법
범위 확장 욕심	일정/복잡도 무한 증가	v5.0 support matrix 밖 요청은 기본 보류
오탐 증가	실전 사용성 저하	confirmed 기준 강화, needs_review 활용
상태 변경 요청 위험	대상 데이터 훼손 가능	scope/policy gate + dry-run/review-required
Semantic Judge 변동성	판정 재현성 저하	deterministic judge 우선 + metadata 기록
Benchmark 과적합	실제 성능 착시	여러 대상군과 blind/holdout 케이스 유지
분석 그래프 폭발	성능/안정성 저하	depth/node/time cap 및 graceful fallback

11. 구현 작업 패키지

ID	작업 패키지	완료 산출물	연결 버전
WP-01	HTTP method/body 모델 확장	endpoint artifact + policy metadata	4.5
WP-02	Auth context / object ID 비교	Auth/BOLA validator + evidence	4.5
WP-03	Validator CLI orchestration	scan/validate/reproduce/	4.5

ID	작업 패키지	완료 산출물	연결 버전
		report 연결	
WP-04	Python interprocedural dataflow	call graph + regression fixtures	4.6
WP-05	JS/TS wrapper/middleware dataflow	framework 별 최소 흐름 추적	4.6
WP-06	Semantic judge layer	judge metadata + needs_review logic	4.6
WP-07	실전 benchmark corpus	dataset manifest + baseline metrics	4.7
WP-08	Stability/regression	CI + timeout/crash test	4.7
WP-09	Release hardening	schema freeze + docs + known limitations	5.0
WP-10	Final acceptance	end-to-end release checklist	5.0

12. 프로젝트 Definition of Done

완료 정의 v5.0은 “취약점을 최대한 많이 찾는 버전”이 아니라, “지원 범위가 명확하고 그 범위 안에서는 안전하게 탐지 - 검증 - 재현 - 증거 - 보고를 끝까지 수행하는 버전”이다.

- 새 기능을 추가하지 않아도 프로젝트의 핵심 사용 사례가 성립한다.
- 지원 범위와 미지원 범위가 문서로 구분된다.
- 실전 benchmark 에서 반복되는 P0/P1 실패가 남아 있지 않다.
- finding 이 왜 confirmed 또는 needs_review 인지 evidence 로 설명 가능하다.
- 동일 finding 을 reproduce 명령으로 다시 확인할 수 있다.
- 모든 동적 테스트가 scope/policy 를 통과해야 실행된다.
- v5.0 이후 개선은 실제 실패 사례가 근거일 때만 진행한다.

13. 최종 요약

v4.4.0까지는 LLM/RAG/Agent 보안 자동화의 핵심 축이 상당 부분 완성되어 있다. 남은 작업은 범위를 끝없이 늘리는 것이 아니라, Web/API/Auth 검증과 dataflow 연결부를 보완하고 실제 benchmark 로 품질을 확인한 뒤 기능을 동결하는 것이다.

단계	한 줄 목표
4.5	실제 요청과 validator 를 연결한다.
4.6	정적 분석과 판정을 실용 수준으로 강화한다.
4.7	실전 데이터로 실패 원인을 측정하고 고친다.
5.0	기능을 동결하고 안정적인 최종 버전을 만든다.
5.0.x	실전 테스트로 확인된 문제만 패치한다.

최종 원칙 “기능 추가”가 아니라 “실전에서 반복되는 실패를 줄이는 것”을 v5.0 이후의 유일한 개선 기준으로 삼는다.